

Scripture Live:

search, then follow

Turning a single-file demo into a two-corpus product — Bible and Quran — that tracks a live reader hands-free, with no operator clicking ahead to guess where they are going.

TODAY 155 verses, Chrome only	TARGET CORPORA 31,102 + 6,236	STACK Vite · TS · transformers.js	BROWSERS all four engines
CRITICAL PATH Arabic recitation ASR	TO FIRST GATE ~1 week		

The reframe that drives everything

The demo *searches*. The product *searches once, then tracks*.

Scanning the whole corpus on every utterance works at 155 verses because almost any match lands somewhere plausible. At 31,102 it collapses — `and it came to pass` appears roughly 450 times in the KJV, so a pure overlap scorer picks noise with confidence.

Acquire position once, then lock and follow forward. If the reader is at Psalm 23:3, the next utterance is almost certainly 23:4 — the search space drops from 31,102 candidates to about twelve. **Context carries the weight the audio cannot**, which is what makes imperfect speech recognition survivable. Build this early; most of the architecture follows from it.

SEARCH · ALL 31,102



7 plausible false hits · 1 correct

↓ lock on to position

TRACK · WINDOW OF 12



every false hit now sits outside the window

Tracking is a small forced-alignment problem, not a search problem. Keep a periodic global check running underneath to catch genuine jumps.

Stack and browser support

Cross-browser is a consequence of owning the recognition

The Web Speech API is the only thing making the current demo Chrome-only. This is not a polish problem to be solved by feature detection, because two of the four engines have no implementation to detect.

	webkitSpeechRecognition	getUserMedia	WEBASSEMBLY
Chrome / Edge	yes — audio streamed to Google or Azure	yes	yes
Safari desktop	partial, unreliable for continuous use	yes	yes
iOS Safari	partial, heavily restricted	yes	yes
Firefox	never shipped	yes	yes

Firefox exposes a `media.webspeech.recognition.enable` preference with no backend behind it, and that is not changing. The other two columns are universal. **Capture the audio directly and run recognition in the page, and the browser question disappears.** Cross-browser support is not a later feature — it falls out of owning the recognition step.

That step has to be owned regardless: Arabic recitation will never work on a conversational model trained for Modern Standard Arabic, so Phase 3 forces it. Doing it in Phase 1 avoids solving browser support twice.

transformers.js is the specific route — Whisper through ONNX Runtime Web, using WebGPU where available (Chrome, Edge, Safari 18+, with Firefox rolling out) and falling back to WASM everywhere else. Quantized `whisper-tiny` is roughly 40 MB and `whisper-base` roughly 80 MB, cached in IndexedDB after first load. That buys every browser, offline operation, no per-minute cost, and no audio leaving the building — which matters when the audio is a congregation.

Keep the existing Web Speech path as an opportunistic fast lane on Chrome. It is already written and costs nothing. It must simply stop being the only path. iOS is the weakest target: HTTPS and a user gesture are required, background audio is restricted, and WASM is slow on older iPhones. It works, but budget real time for it if congregant phones are in scope.

Build stack

Vite and TypeScript, no UI framework, through Phase 2. TypeScript settles the build-step question before it is asked. Vite is near-zero configuration and provides Web Worker imports, WASM loading, and code splitting — all three of which this app needs specifically. That is the floor, not over-engineering.

No UI framework yet: through Phase 2 the product is one screen and roughly 1,500 lines, and a framework earns nothing against that. **Add Svelte at Phase 4**, when views multiply — setup, live, history, settings. Svelte rather than React because it compiles away, and the

byte budget is already committed to a ~1.5 MB corpus plus a ~40 MB model; also because the transcript repaints several times per second and a virtual DOM is pure overhead on that path. Deployment stays Vercel static and unchanged through Phase 3.

The one thing that breaks static-only hosting

Streaming server-side recognition needs a WebSocket, a long-lived process, and — on the GPU route — a GPU. Vercel functions provide none of these, so that path means a separate service on Fly.io, Railway, or a plain GPU VPS.

Which is itself an argument for the in-browser route: transformers.js keeps the product on pure static hosting through Phase 3. A server becomes unavoidable only when licensed translations arrive, since copyrighted text cannot be shipped to the client — a different reason, in a later phase.

Five phases

Ordered by dependency, not preference — each phase exists because the one after it is unsafe to start without it. Every phase ends at a gate you can measure rather than eyeball.



~1 week

De-risk the unknowns

Two questions can kill this product. Answer both before writing any product code.

- **Spike A — archaic English.** Web Speech API is trained on conversational speech; the KJV is *whithersoever* and *peradventure*. Record 20 readings, measure word error rate.
- **Spike B — Quranic recitation.** The real risk. The `ar-SA` model is trained on Modern Standard Arabic news and conversation, while recitation is Classical Arabic with tajweed: elongations, melodic delivery, specific articulation. Expect failure, and find out in week one rather than month four.
- **Build the eval harness now.** Audio in, predicted verse ID out, compared against ground truth. Everything downstream gets measured against it, so no change is ever a guess.
- **Free labelled data exists for both.** EveryAyah.com serves verse-by-verse recitation from many qaris — each file *is* one ayah with a known ID, roughly 6,236 labelled clips per reciter. LibriVox carries public-domain KJV audio of known text.

GATE A measured word error rate for both corpora, and a harness that scores a run automatically. If Spike B fails badly, Quran needs a fine-tuned model — a different budget and timeline, and better known now.

01

~2–3 weeks

Make one corpus actually work

Full KJV, 31,102 verses. Still static, still no framework — the app is genuinely small at this stage.

- **Index.** Ship plain text (~4.5 MB, ~1.5 MB gzipped) and build the inverted index in a Web Worker at load. Shipping a prebuilt index buys about 200 ms and costs a format to maintain.
- **Scoring.** BM25, not raw overlap. It weights rare terms — precisely what the current scorer lacks. Pull candidates from the inverted index so you score roughly 150 verses instead of 31,102.
- **Tracking.** Position plus confidence. Locked, score only a window around the current verse; advance when the next verse outscores it; unlock when a global check beats the window by a margin for several consecutive frames. It is a small HMM — resist over-building it.
- **Fix the rolling transcript.** Use `e.resultIndex` and keep the last ~15 words. The present code rebuilds the entire session transcript on every event, which drives the match score below threshold after roughly 60–80 spoken content words. Tracking needs the fix anyway.
- **Put ASR behind an interface** so Phase 3 is a swap, not a rewrite.

GATE Harness reports precision above 95% on high-confidence matches across LibriVox samples. A number, not a feeling.

02

~1 week

Abstract the corpus

Both scriptures are ordered collections of numbered units, so they share one

engine. They differ in ways that will cost you if collapsed.

	BIBLE	QURAN
Canonical text	the translation, for English readers	the Arabic; translation is secondary
Direction	left to right	right to left
Mode	reading	recitation

- Shared flat verse index, with a per-corpus reference formatter, normalizer, and renderer.
- **One engine, two adapters.** Do not build two apps; do not pretend they are the same thing either.

GATE The KJV path still passes every Phase 1 measurement after the refactor.

03

~3–4 weeks

Quran

Scope here depends entirely on what Phase 0 measured. Everything else is ready for it.

- **Text.** Tanzil.net, verified Uthmani. It ships both a simple variant — diacritics stripped, alif, ya and ta-marbuta normalized — and the full Uthmani. Match on simple, display Uthmani, write no normalization code of your own.
- **Rendering.** Right to left, a proper Arabic face with full harakat, optional transliteration and paired translation.
- **Recognition.** Ranked by what Phase 0 found: Web Speech API (free, weak on recitation) → whisper.cpp in WASM (offline, ~75 MB model, better but still soft on tajweed) → server-side Whisper (strongest general model, real cost) → a model fine-tuned on open recitation datasets (best here, real training effort).

GATE Correct ayah identified on held-out EveryAyah clips across at least three reciters — different voices, not three takes of one.

Product shell

Accounts, saved sessions, projection output, offline mode.

- This is where a backend and a framework finally earn their place — **not before**. Through Phase 3 the app runs to roughly 1,500 lines and vanilla is the correct answer.
- Licensed translations force a server regardless, since you cannot ship copyrighted text to the client.

GATE A full service or khutbah followed end to end without losing lock or needing a hand on the keyboard.

Two things that will bite

Licensing is a gating dependency, not a detail

The KJV is public domain in the US; UK Crown copyright is perpetual but not practically enforced abroad. Quran Arabic via Tanzil is clean. But the NIV, ESV and NLT are heavily copyrighted, cost real money, and cannot be shipped to the client — which forces a server the moment you add one. Quran translations vary by translator. **Design for pluggable translations from day one and launch on public domain only**, so the question is already answered when someone asks for the NIV.

A wrong verse is a trust failure, not a bug

The wrong verse on screen during a service is embarrassing. Wrong Quran text is offensive and can end the product outright. So: **precision over recall, always**. Never render a low-confidence match as though it were certain — show the reference, show alternatives, keep manual override one tap away. The existing confidence bar is the right instinct; make it authoritative rather than decorative.

Where you are actually differentiated

Church projection belongs to ProPresenter and EasyWorship. Quran recitation recognition belongs to Tarteel. The wedge is the thing neither does well: **real-time, hands-free, no**

operator. Nobody clicking ahead trying to guess where the preacher is going next. That is genuinely underserved — build toward it rather than toward a general scripture viewer.

Still open

Who holds the screen?

Phases 0 through 2 are identical either way, so this is not blocking — but it reshapes Phase 4 substantially, and it is the next thing worth settling. Same engine, three different products.

congregant following along

operator running projection

student drilling hifz

Scripture Live · roadmap drafted from a review of the current prototype

Verse counts verified against the shipped index: 26 books, 155 verses today · KJV 66 books, 1,189 chapters, 31,102 verses · Quran 114 surahs, 6,236 ayat